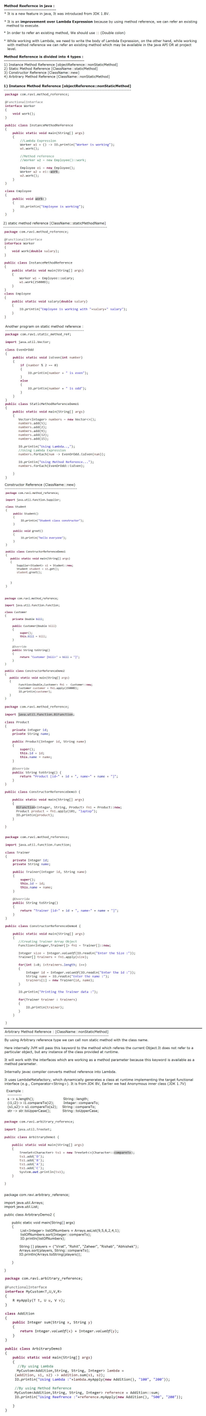

Method Reefernce in java :

* It is a new feature in java, It was introduced from JDK 1.8V.

* It is an **improvement over Lambda Expression** because by using method reference, we can refer an existing method to execute.

* In order to refer an existing method, We should use :: (Double colon)

* While working with Lambda, we need to write the body of Lambda Expression, on the other hand, while working with method reference we can refer an existing method which may be available in the java API OR at project level.

Method Reference is divided into 4 types :

1) Instance Method Reference [objectReference::nonStaticMethod]
2) Static Method Reference [ClassName::staticMethod]
3) Constructor Reference [ClassName::new]
4) Arbitrary Method Reference [ClassName::nonStaticMethod]

1) Instance Method Reference [objectReference::nonStaticMethod]

```
package com.ravi.method_reference;

@FunctionalInterface
interface Worker
{
    void work();
}

public class InstanceMethodReference
{
    public static void main(String[] args)
    {
        //Lambda Expression
        Worker w1 = () -> IO.println("Worker is working");
        w1.work();

        //Method reference
        //Worker w2 = new Employee()::work;

        Employee e1 = new Employee();
        Worker w2 = e1::work;
        w2.work();
    }
}

class Employee
{
    public void work()
    {
        IO.println("Employee is working");
    }
}
```

2) static method reference [ClassName::staticMethodName]

```
package com.ravi.method_reference;

@FunctionalInterface
interface Worker
{
    void work(double salary);
}

public class InstanceMethodReference
{
    public static void main(String[] args)
    {
        Worker w1 = Employee::salary;
        w1.work(25000D);
    }
}
class Employee
{
    public static void salary(double salary)
    {
        IO.println("Employee is working with "+salary+" salary");
    }
}
```

Another program on static method reference :

```
package com.ravi.static_method_ref;

import java.util.Vector;

class EvenOrOdd
{
    public static void isEven(int number)
    {
        if (number % 2 == 0)
        {
            IO.println(number + " is even");
        }
        else
        {
            IO.println(number + " is odd");
        }
    }
}
public class StaticMethodReferenceDemo1
{
    public static void main(String[] args)
    {
       Vector<Integer> numbers = new Vector<>();
       numbers.add(5);
       numbers.add(2);
       numbers.add(9);
       numbers.add(12);
       numbers.add(15);

       IO.println("Using Lambda..,");
       //Using Lambda Expression
       numbers.forEach(num -> EvenOrOdd.isEven(num));

       IO.println("Using Method Reference...");
       numbers.forEach(EvenOrOdd::isEven);

    }
}
```

Constructor Reference (ClassName::new)

```
package com.ravi.method_reference;

import java.util.function.Supplier;

class Student
{
    public Student()
    {
        IO.println("Student class constructor");
    }

    public void greet()
    {
        IO.println("hello everyone");
    }
}

public class ConstructorReferenceDemo1
{
  public static void main(String[] args)
  {
     Supplier<Student> s1 = Student::new;
     Student student = s1.get();
     student.greet();


  }
}
```

```
package com.ravi.method_reference;

import java.util.function.Function;

class Customer
{
    private Double bill;

    public Customer(Double bill)
    {
        super();
        this.bill = bill;
    }

    @Override
    public String toString()
    {
        return "Customer [bill=" + bill + "]";
    }
}

public class ConstructorReferenceDemo2
{
  public static void main(String[] args)
  {
      Function<Double,Customer> fn1 =  Customer::new;
      Customer customer = fn1.apply(15000D);
      IO.println(customer);
  }
}
```

```
package com.ravi.method_reference;

import java.util.function.BiFunction;

class Product
{
    private Integer id;
    private String name;

    public Product(Integer id, String name)
    {
        super();
        this.id = id;
        this.name = name;
    }

    @Override
    public String toString() {
        return "Product [id=" + id + ", name=" + name + "]";
    }
}

public class ConstructorReferenceDemo3 {

    public static void main(String[] args)
    {
      BiFunction<Integer, String, Product> fn1 = Product::new;
      Product product = fn1.apply(101, "laptop");
      IO.println(product);
    }

}
```

```
package com.ravi.method_reference;

import java.util.function.Function;

class Trainer
{
    private Integer id;
    private String name;

    public Trainer(Integer id, String name)
    {
        super();
        this.id = id;
        this.name = name;
    }

    @Override
    public String toString()
    {
        return "Trainer [id=" + id + ", name=" + name + "]";
    }
}

public class ConstructorReferenceDemo4 {

    public static void main(String[] args)
    {
       //Creating Trainer Array Object
       Function<Integer,Trainer[]> fn1 = Trainer[]::new;

       Integer size = Integer.valueOf(IO.readln("Enter the Size :"));
       Trainer[] trainers = fn1.apply(size);

       for(int i=0; i<trainers.length; i++)
       {
           Integer id = Integer.valueOf(IO.readln("Enter the id :"));
           String name = IO.readln("Enter the name :");
           trainers[i] = new Trainer(id, name);
       }

       IO.println("Printing the Trainer data :");

       for(Trainer trainer : trainers)
       {
           IO.println(trainer);
       }
    }

}
```

Arbitrary Method Reference : [ClassName::nonStaticMethod]

By using Arbitrary reference type we can call non static method with the class name.

Here internally JVM will pass this keyword to the method which referes the current Object.It does not refer to a particular object, but any instance of the class provided at runtime.

It will work with the interfaces which are working as a method parameter because this keyword is available as a method parameter.

Internally javac compiler converts method reference into Lambda.

It uses LambdaMetafactory, which dynamically generates a class at runtime implementing the target functional interface (e.g., Comparator<String>). It is from JDK 8V, Earlier we had Anonymous inner class (JDK 1.7V)

Example :

s -> s.length();	String::length;
(i1,i2)-> i1.compareTo(i2);	Integer::compareTo;
(s1,s2)-> s1.compareTo(s2);	String::compareTo;
str -> str.toUpperCase();	String::toUpperCase;

```
package com.ravi.arbitrary_reference;

import java.util.TreeSet;

public class ArbitraryDemo1 {

    public static void main(String[] args)
    {
        TreeSet<Character> ts1 = new TreeSet<>(Character::compareTo);
        ts1.add('D');
        ts1.add('B');
        ts1.add('A');
        ts1.add('C');
        System.out.println(ts1);
    }

}
```

```
package com.ravi.arbitrary_reference;

import java.util.Arrays;
import java.util.List;

public class ArbitraryDemo2 {

    public static void main(String[] args)
    {
         List<Integer> listOfNumbers = Arrays.asList(9,5,6,2,4,1);
         listOfNumbers.sort(Integer::compareTo);
         IO.println(listOfNumbers);

        String [] players = {"Virat", "Rohit", "Zaheer", "Rishab", "Abhishek"};
        Arrays.sort(players, String::compareTo);
        IO.println(Arrays.toString(players));

    }

}
```

```
package com.ravi.arbitrary_reference;

@FunctionalInterface
interface MyCustom<T,U,V,R>
{
    R myApply(T t, U u, V v);
}

class Addition
{
    public Integer sum(String x, String y)
    {
        return Integer.valueOf(x) + Integer.valueOf(y);
    }
}

public class ArbitraryDemo3
{
    public static void main(String[] args)
    {
       //By using Lambda
      MyCustom<Addition,String, String, Integer> lambda =
     (addition, s1, s2) -> addition.sum(s1, s2);
     IO.println("Using Lambda :"+lambda.myApply(new Addition(), "100", "200"));

     //By using Method Reference
     MyCustom<Addition,String, String, Integer> reference = Addition::sum;
     IO.println("Using Reefrence :"+reference.myApply(new Addition(), "500", "200"));

    }
}
```